

LAPORAN PRAKTIKUM
TEKNOLOGI IoT (TUGAS 3)



Disusun Oleh :

Nama : Thesa Septi Listiana

Kelas : TRI 3B

NPM : 25782057

Dosen Pembimbing :

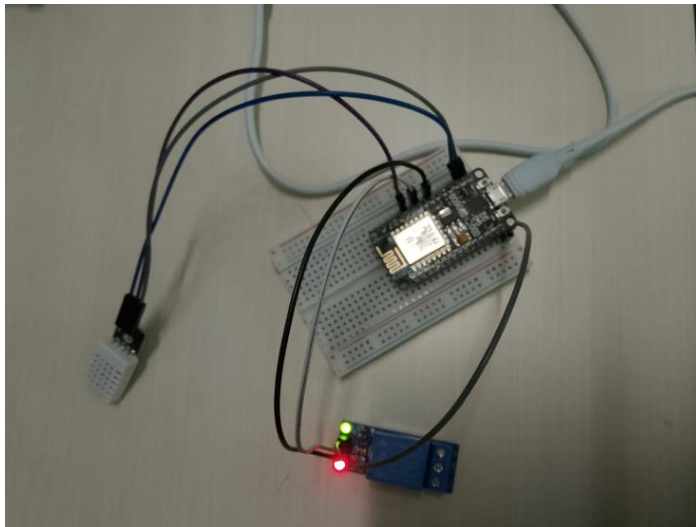
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom

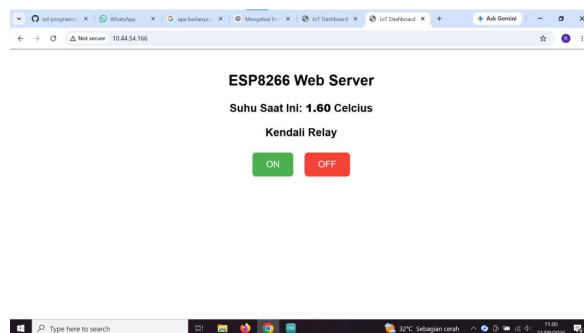
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

3.5.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML



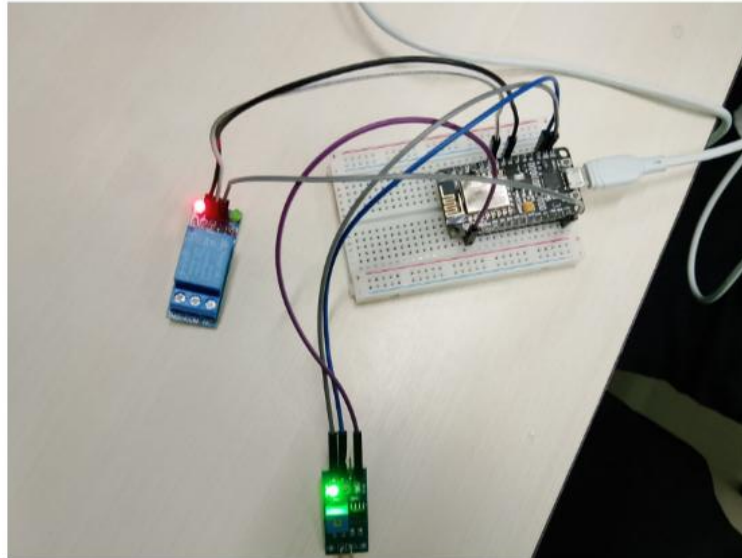
```
sketch_sep21a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E Module)
Sketchbook
sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Yandri";
6 const char* password = "12345678";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18
19 </head>
20 <body>
21
22 </body>
23 </html>
24 )rawliteral";
25
26 void setup() {
27   Serial.begin(115200);
28   WiFi.begin(ssid, password);
29   while (WiFi.status() != WL_CONNECTED) {
30     delay(5000);
31     Serial.print("Connecting to WiFi... ");
32   }
33   Serial.println("Connected to WiFi");
34   server.on("/", []() {
35     Serial.println("Index page");
36     server.send(200, "text/html", index_html);
37   });
38   server.on("/api/relay", []() {
39     Serial.println("Relay control");
40     digitalWrite(relayPin, !digitalRead(relayPin));
41     server.send(200, "text/plain", "Relay status: " + String(digitalRead(relayPin)));
42   });
43   server.begin();
44   Serial.println("Web server started");
45 }
46
47 void loop() {
48   server.handleClient();
49   delay(1000);
50   float temp = dht.temperature();
51   float hum = dht.humidity();
52   Serial.print("Temp: ");
53   Serial.print(temp);
54   Serial.print(" Hum: ");
55   Serial.print(hum);
56   Serial.println();
57 }
```



Penjelasnya: Praktikum 1: NodeMCU sebagai Web Server adalah sistem di mana NodeMCU terhubung ke jaringan Wi-Fi (SSID "Yandri") dan bertindak sebagai server lokal yang menampilkan data suhu DHT11 serta tombol kontrol relay melalui alamat IP 10.44.54.166 saat diakses via *browser*.

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

1. menjadi sebuah server (thesa)



```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4
5  const char* ssid = "Nizar";
6  const char* password = "12345678";
7
8  const char* serverName = "http://10.88.207.97/relay/on";
9  const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

.....

IP Address Server Anda:
10.44.54.166

.....

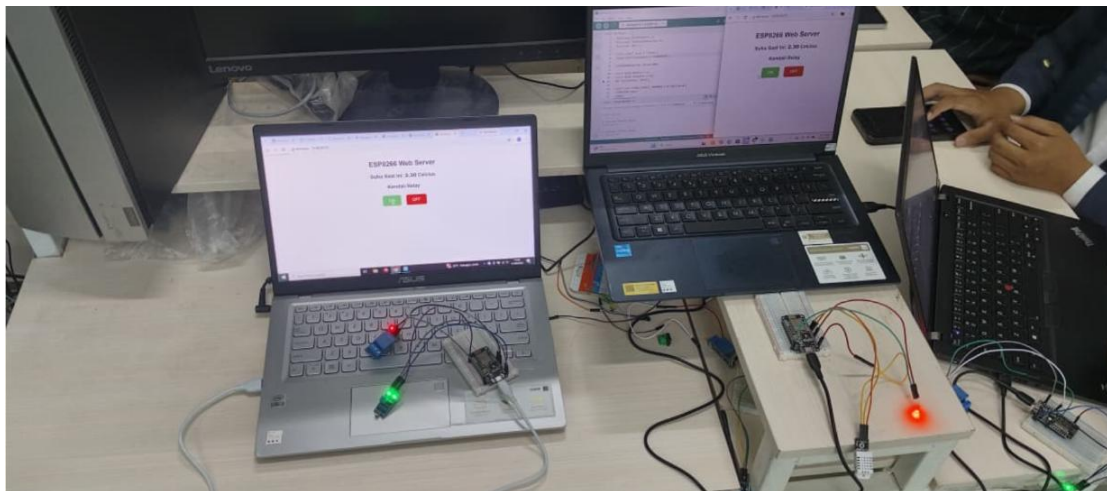
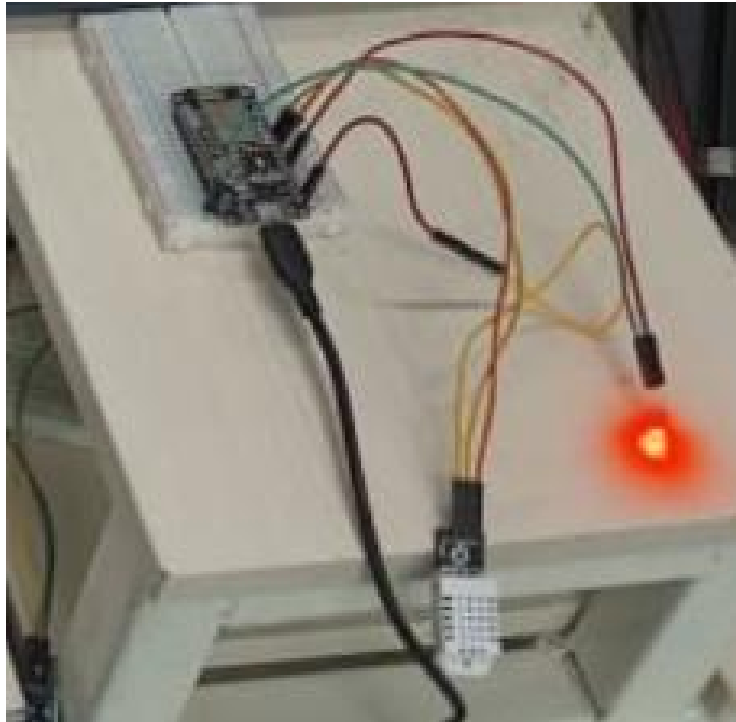
IP Address Server Anda:
10.44.54.166

.....

Client Terhubung ke Wi-Fi!

Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan) adalah sistem di mana NodeMCU bertindak sebagai klien yang terhubung ke jaringan Wi-Fi (SSID "Nizar"), membaca nilai dari sensor cahaya (LDR) pada pin analog A0, lalu secara aktif mengirimkan permintaan HTTP (seperti *GET request* ke http://10.88.207.97/relay/on) untuk mengontrol perangkat lain secara otomatis berdasarkan hasil pembacaan sensor tersebut.

2. Sebagai client (Yandri)



Penjelasannya :Berdasarkan gambar di atas, sistem bekerja sebagai **client** di mana laptop atau perangkat yang membuka *browser* bertindak sebagai pihak yang mengakses halaman web server (menampilkan menu "ESP8266 Web Server" dengan tombol ON/OFF serta informasi suhu) untuk memantau sensor dan mengontrol relay dari jarak jauh secara nirkabel.

3.6 Latihan

1. Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, dan bukan mode Access Point (AP)?

➤ *Jawaban:* Kita menggunakan mode Station (STA) karena NodeMCU bertindak sebagai perangkat klien yang bergabung ke jaringan router Wi-Fi atau *hotspot* yang sudah ada. Jika menggunakan mode Access Point (AP), NodeMCU justru harus memancarkan sinyal Wi-Fi sendiri, sehingga perangkat tidak bisa terhubung ke jaringan lokal utama secara bersamaan.

2. Pada Praktikum 1, lakukan modifikasi Web Server untuk menampilkan kelembapan (%HUMIDITY%) serta ubah tombol menjadi satu tombol dinamis tunggal (*toggle button*) berdasarkan status Relay. Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang diubah!

➤ *Jawaban:*

```
C++
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top:
50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius:
8px; margin: 10px; cursor: pointer;}
    .btn-on { background-color: #4CAF50; color: white; border:
none; }
    .btn-off { background-color: #f44336; color: white; border:
none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
```

```

        <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
    </div>
    <h2>Kendali Relay</h2>
    %RELAY_BUTTON%
</body>
</html>
)rawliteral";
void handleRoot() {
    String html = index_html;
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (isnan(t)) {
        html.replace("%TEMPERATURE%", "--");
    } else {
        html.replace("%TEMPERATURE%", String(t));
    }

    if (isnan(h)) {
        html.replace("%HUMIDITY%", "--");
    } else {
        html.replace("%HUMIDITY%", String(h));
    }

    int relayState = digitalRead(relayPin);
    if (relayState == LOW) {
        html.replace("%RELAY_BUTTON%", "<a
href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN
(ON)</button></a>");
    } else {
        html.replace("%RELAY_BUTTON%", "<a
href=\"/relay/off\"><button class=\"btn-off\">MATIKAN
(OFF)</button></a>");
    }

    server.send(200, "text/html", html);
}

```

3. (Analisis Kode Client) Pada Praktikum 2, apa yang akan dicetak oleh Serial Monitor Client jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (offline)?

➤ *Jawaban:* Serial Monitor Client akan mencetak kode error negatif dari HTTPClient, yaitu -1 (atau pesan kegagalan koneksi seperti *connection refused*), karena klien tidak berhasil menjangkau atau terhubung ke alamat IP server tujuan.

4. Latihan 4 (Pengembangan Algoritma) Modifikasi conditional block if di Praktikum 2 sehingga Client mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35°C. Tulis baris algoritmanya!

➤ *Jawaban:*

```
C++
float t = dht.readTemperature(); // Membaca suhu dari sensor DHT
if (!isnan(t) && t > 35.0) {
    HTTPClient http;
    http.begin(wifiClient, serverName);
    int httpStatusCode = http.GET();

    Serial.print("Suhu > 35° C! HTTP Response code: ");
    Serial.println(httpStatusCode);
    http.end();
} else {
    Serial.println("Suhu normal (<= 35° C), tidak mengirim request.");
}
```

3.7 Tugas: Pengembangan Web Server Cerdas & Analisis Arsitektur

1. Modifikasi variabel `index_html` dengan menyisipkan tag Meta Refresh HTML (`<meta http-equiv="refresh" content="5">`) di dalam `<head>` agar halaman memuat ulang otomatis setiap 5 detik. Analisis Arsitektur: Apa kelemahan visual/interaksi (*flickering*) saat menggunakan metode Meta Refresh konvensional untuk sistem *Real-Time Dashboard*?

➤ *Jawaban:*

Modifikasi Kode Server (Auto-Refresh):

```
C++
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <meta http-equiv="refresh" content="5"> <!-- Auto-Refresh
tiap 5 detik -->
    <title>IoT Dashboard</title>
    <!-- style CSS -->
</head>
)rawliteral";
```

Analisis Kelemahan Arsitektur:

- a. Visual Kedipan (*Flickering*): Halaman web mengalami *full-page reload* secara berkala, menyebabkan layar browser berkedip putih setiap 5 detik yang menurunkan kenyamanan visual.
- b. Pemborosan Bandwidth: Browser mengunduh ulang seluruh struktur kode HTML beserta asetnya dari awal, bukan sekadar memperbarui data suhunya saja.
- c. Potensi Bentrokan (*Race Condition*): Jika tombol relay diklik tepat bersamaan dengan waktu *refresh* halaman, perintah kontrol bisa gagal terkirim karena halaman mendadak dimuat ulang.